

Lección 13: Observability Strategy

Testear Menos vs Observar Más

Agenda

- El Dilema: Testing vs Observability
 - Framework de Decisión (4 Preguntas)
 - Matriz por Etapa del Proyecto
 - Casos de Estudio Reales
 - ROI de Observability
-

¿Qué es ROI?

ROI (Return on Investment): Retorno sobre la inversión

Fórmula:

$$\text{ROI} = (\text{Beneficio} - \text{Costo}) / \text{Costo}$$

Ejemplo práctico:

Escribir tests: 150 minutos (costo)

Feature adoption: 5% usuarios (beneficio bajo)

ROI: NEGATIVO (-26h desperdiciadas)

Setup observability: 10 minutos (costo)

Descubrir adoption temprano: 5% (beneficio: evitar 140 min)

ROI: POSITIVO (+140 min ahorrados)

En esta lección: Medimos ROI de testing vs observability para

tomar decisiones inteligentes

La Pregunta Central

¿Cuándo debemos **testear exhaustivamente** vs **observar en producción**?

Respuesta: Depende del **ROI** y la **fase del proyecto**

Idea Clave:

- Tests predicen problemas conocidos
 - Observability descubre problemas desconocidos
-

Framework: 4 Preguntas Clave

1. ¿Conoces el comportamiento esperado?

✅ **TESTEAR:** Comportamiento definido

- Cálculos financieros
- Validaciones de negocio
- APIs con contratos

🔍 **OBSERVAR:** Comportamiento incierto

- ¿Qué features usan usuarios?
 - ¿Dónde abandonan el flujo?
 - ¿Cómo navegan realmente?
-

Framework (cont.)

2. ¿Cuál es el costo de fallar?

✅ **TESTEAR:** Costo alto

- Transacciones financieras
- Sistemas críticos de seguridad
- Lógica core del negocio

- 🔍 OBSERVAR: Costo bajo-medio
 - Mejoras de UX/UI
 - Features experimentales
 - Optimizaciones de performance
-

Framework (cont.)

3. ¿Qué tan estable es el requisito?

- ✅ TESTEAR: Requisitos estables
 - Reglas de negocio consolidadas
 - APIs públicas maduras
 - Sistemas legacy establecidos

- 🔍 OBSERVAR: Requisitos cambian
 - Features nuevas en exploración
 - Experimentos A/B
 - Comportamiento incierto
-

Framework (cont.)

4. ¿Puedes simular el escenario real?

- ✅ TESTEAR: Simulación efectiva
 - Lógica pura (input → output)
 - Casos de borde conocidos
 - Integraciones con mocks

- 🔍 OBSERVAR: Simulación limitada
 - Comportamiento de usuario real
 - Performance con datos producción
 - Interacciones complejas del sistema
-

Matriz por Etapa del Proyecto

Etapa MVP:

Prioridad: OBSERVAR > TESTEAR

60% Observability

- Seguimiento de errores
- Recorridos de usuario
- Métricas de adopción

40% Testing

- Tests de humo
- Happy path (camino feliz)
- Casos críticos

¿Por qué? No sabemos qué funciona aún

Matriz (cont.)

Etapa de Crecimiento:

Prioridad: BALANCEADO

50% Observability

- Performance
- Embudos de conversión
- Pruebas A/B

50% Testing

- Pruebas de regresión
- Tests de integración

¿Por qué? Optimizar manteniendo estabilidad

Matriz (cont.)

Etapa de Escala:

Prioridad: TESTEAR > OBSERVAR

70% Testing

- Coverage exhaustivo
- Pruebas de contrato
- Pruebas de performance

30% Observability

- Monitoreo
- Alertas
- Inteligencia de negocio

¿Por qué? Estabilidad es crítica

Caso 1: Feature de Descuentos

✗ Mal enfoque - Solo Testing:

```
// 50 tests cubriendo todos edge cases
it('should apply 15% discount for $100+ orders')
it('should apply 10% bulk discount for 5+ items')
// ... 47 tests más
```

Resultado:

- 40 horas escribiendo tests
 - 2 bugs encontrados
 - No sabemos si usuarios usan descuentos
-

Caso 1 (cont.)

✓ Buen enfoque - Testing + Observability:

```
// 5 tests para casos críticos
it('should calculate discounts correctly')

// Observabilidad para uso real
captureBusinessMetric('discount.applied', {
  discount_type: 'bulk|order|seasonal',
  user_segment: 'new|returning|premium'
})
```

Resultado:

- 8 horas implementando
 - Descubrimos: nadie usa descuentos estacionales
 - Eliminamos feature → ahorro 20 horas
-

Caso 2: Performance de Checkout

✖ Mal enfoque:

```
// Tests con datos sintéticos
it('checkout should complete in under 2 seconds')
```

Resultado:

- Tests pasan en desarrollo
- No detecta problemas reales

✔ Buen enfoque:

```
// Timing real
const endTiming = startBusinessTiming('checkout_process')
// ... lógica
endTiming({ user_type, cart_size, payment_method })
```

Resultado:

- 80% usuarios abandonan en step 3
 - Optimizamos UX → +15% conversion
-

Estrategias Prácticas

Para Features Nuevas (MVP):

```
// 1. Empezar con Observability
captureUserJourney('feature_used', {
  feature: 'new_calculator'
})

// 2. Testear solo Happy Path
it('should calculate correctly for common case')
```

```
// 3. Observar uso real
if (adoption < 5%) {
  // Remover feature en lugar de agregar tests
}
```

Señales de Alerta

► Exceso de Testing:

- Más tiempo tests que features
- Tests que nunca fallan
- 100% coverage en features experimentales
- Tests de implementación, no comportamiento

Cura: Agregar observability, remover tests redundantes

► Falta de Observability:

- No sabes cómo usan tu producto
- Bugs reportados por usuarios, no tu sistema
- Problemas de performance descubiertos accidentalmente

Cura: Agregar métricas de negocio, seguimiento de errores

Ejercicio: Análisis de ROI

Feature “Favoritos” en e-commerce:

Opción A: Testing Primero

- Esfuerzo: 20h tests + 8h feature = 28h
- Resultado: Perfectamente testada, solo 2% uso
- ROI: Negativo (-26h desperdiciadas)

Opción B: Observability Primero

- Esfuerzo: 2h observability + 8h feature = 10h
- Resultado: Descubres que prefieren “Comprar después”

- ROI: Positivo (+18h ahorradas + feature útil)

Ejercicio 1: Testing vs Observability con Framework

Prompt:

Actúa como un arquitecto de software aplicando Framework 4 Preguntas para d

CONTEXTO: Pregunta central: "**¿Cuándo testear exhaustivamente vs observar en producción?**". Respuesta: Depende de ROI y fase del proyecto. Tests predicen problemas conocidos (comportamiento definido). Observability descubre probl desconocidos (comportamiento incierto). Framework 4 Preguntas: 1) ¿Conoces comportamiento esperado? (SÍ → testear), 2) ¿Costo de fallar? (Alto → teste 3) ¿Requisito estable? (SÍ → testear), 4) ¿Simulable en tests? (SÍ → testea 3/4 respuestas SÍ → testear, 3/4 NO → observar. Lo mejor de ambos: combinar + observability para cobertura completa.

TAREA: Analiza función applyPromoCode usando Framework 4 Preguntas y decide

FUNCIÓN A ANALIZAR:

- Nombre: applyPromoCode(code: string, cartTotal: number): number
- Propósito: Aplica códigos promocionales (% descuento)
- Lógica: Valida código → calcula descuento → retorna monto descontado
- Uso: E-commerce checkout flow

FRAMEWORK 4 PREGUNTAS (Responder SÍ/NO + razón):

Pregunta 1: ¿Conoces comportamiento esperado?

- Analizar: ¿Códigos están definidos en DB? ¿Reglas claras?
- Respuesta: [SÍ/NO] – [razón]

Pregunta 2: ¿Costo de fallar es alto?

- Analizar: ¿Afecta dinero? ¿Pérdida de revenue? ¿Impacto crítico?
- Respuesta: [SÍ/NO] – [razón]

Pregunta 3: ¿Requisito estable?

- Analizar: ¿Reglas de descuento cambian frecuentemente?
- Respuesta: [SÍ/NO] – [razón]

Pregunta 4: ¿Simulable en tests?

- Analizar: ¿Inputs/outputs conocidos? ¿Casos predecibles?
- Respuesta: [SÍ/NO] – [razón]

DECISIÓN REQUERIDA:

- Contar SÍ: [X/4]
- Estrategia: [TESTEAR/OBSERVAR/HÍBRIDA]
- Justificación: [2-3 oraciones basadas en respuestas]

SI TESTEAR:

- Especificar tests requeridos (unitarios, edge cases)

SI OBSERVAR:

- Especificar métricas a observar

SI HÍBRIDA:

- Testing: [qué testear]
- Observability: [qué observar]

VALIDACIÓN: Decisión debe ser TESTEAR (función crítica de dinero)

Aprende: Framework sistemático transforma decisiones

subjetivas en objetivas

Ejercicio 2: Calcular ROI Testing vs Observability

Prompt:

Actúa como un product engineer calculando ROI para decidir entre testing vs

CONTEXTO: ROI (Return on Investment – retorno sobre inversión) mide si inve de tiempo vale la pena. Fórmula: $ROI = (Beneficio - Costo) / Costo$. Testing Primero: alto costo inicial (150 min escribir tests), beneficio solo si fea se usa. Observability Primero: bajo costo inicial (10 min setup), descubre adopción real antes de invertir en tests. Features experimentales: adopción desconocida (puede ser 2% o 80%). Regla pragmática: adopción > 50% → testea exhaustivamente, adopción < 20% → considerar remover/mejorar.

TAREA: Calcula ROI de ambas opciones para feature experimental con 5% adopc

ESCENARIO:

- Feature: Nueva funcionalidad "Quick Reorder" (reordenar rápido)
- Adopción esperada: DESCONOCIDA (puede ser 2% o 80%)
- Adopción real: 5% (descubierto después)

OPCIÓN A – TESTING PRIMERO:

- Esfuerzo: 5 edge cases × 30 min/test = 150 min
- Tests escritos: 5 tests comprensivos
- Feature lanzada con tests completos
- Adopción descubierta: 5% de usuarios

OPCIÓN B – OBSERVABILITY PRIMERO:

- Esfuerzo: Setup Sentry + tracking = 10 min
- Feature lanzada con observability
- Adopción descubierta: 5% de usuarios
- Decisión post-observability: ¿Vale la pena invertir 150 min en tests?

CÁLCULO ROI REQUERIDO:

Opción A (Testing Primero):

- Costo: [tiempo en min]
- Beneficio: [valor según adopción real 5%]
- ROI: [positivo/negativo + cantidad]
- Conclusión: [¿valió la pena?]

Opción B (Observability Primero):

- Costo: [tiempo en min]
- Beneficio: [tiempo ahorrado al descubrir baja adopción temprano]
- ROI: [positivo/negativo + cantidad]
- Conclusión: [¿valió la pena?]

ANÁLISIS COMPARATIVO:

- Diferencia ROI: [A vs B]
- Decisión óptima: [cuál opción]
- Regla extraída: [cuándo usar cada enfoque]

VALIDACIÓN: Opción B debe tener ROI superior (+140 min ahorrados)

Aprende: Observability primero en features inciertas

maximiza ROI minimizando desperdicio

Puntos Clave

1. **No hay regla única:** Balance depende del contexto
2. **ROI sobre Coverage:** Impacto de negocio > métricas vanidosas
3. **Evoluciona:** MVP → Crecimiento → Escala requieren enfoque diferente
4. **Observa primero:** En features nuevas, observar > testear

5. **Testea lo crítico:** En sistemas maduros, testear > observar
6. **Métricas de negocio:** Mide lo que importa al negocio
7. **Datos de usuarios reales:** Datos de producción > tests sintéticos